

Ensemble Methods: Boosting vs. Bagging

Final Project Report

Davud Gurbanov, Gurban Burjuzada, Murad Asgarov

AI Academy, National AI Center

Spring 2026

Team StarPlatinum

Abstract—This paper presents an empirical comparison of two ensemble learning paradigms—boosting and bagging—using decision-tree base learners. We implement a Decision Tree classifier, AdaBoost (with decision stumps), and a Random Forest from scratch. Controlled experiments on four real-world datasets investigate scaling behaviour, noise robustness, and bias-variance decomposition. Unsupervised analysis (PCA, K-Means, DBSCAN) supplements the visual exploration of the data. Our findings show that AdaBoost achieves lower bias and excels on clean, low-noise datasets, while Random Forest demonstrates superior robustness to label noise and better performance on high-dimensional multi-class problems. On imbalanced data, both methods achieve high accuracy but struggle with minority-class recall.

Deliverables: In addition to this report, we provide the complete source code (tagged `v1.0-final`), a professional slide deck for the oral defense, and a separate contribution report detailing each team member's specific work.

I. INTRODUCTION

The central question of this project is: “*Under what conditions does boosting outperform bagging, and vice versa, and why?*” To answer it, we implement the core algorithms from scratch and evaluate them on datasets of varying characteristics.

Boosting and bagging represent two fundamentally different strategies for ensemble construction. Boosting (AdaBoost) iteratively focuses on misclassified samples, reducing bias at the risk of overfitting to noise. Bagging (Random Forest) averages independent bootstrapped models, reducing variance while maintaining stable bias. These theoretical differences manifest in practice: boosting tends to achieve lower training error but may generalise poorly under label noise, while bagging provides more robust predictions through variance reduction at a slight bias cost.

The project implements four modules: a CART Decision Tree with exhaustive split search, an AdaBoost classifier using decision stumps (SAMME variant), a Random Forest with bootstrap aggregation and feature sub-sampling, and an unsupervised analysis pipeline (PCA, K-Means, DBSCAN). All algorithms are implemented from scratch using only NumPy, with sklearn used solely as a reference baseline for verification (within the allowable 2% tolerance).

The remainder of the paper is structured as follows. Section II reviews essential literature. Section III describes our implementations. Section IV details the experimental setup and Section V presents the results. Section VI discusses

the findings, and Section VII concludes. The accompanying slide deck and contribution report are submitted separately as required by the project brief.

II. RELATED WORK

A. Decision Trees

Decision trees recursively partition the feature space to minimise impurity [1]. CART [2] uses Gini impurity or entropy as splitting criteria, creating binary trees that are interpretable and computationally efficient.

B. Boosting

AdaBoost [3] iteratively re-weights samples, giving higher importance to misclassified instances. The weight update rule is $\alpha_m = \ln \frac{1-\epsilon_m}{\epsilon_m}$ for binary classification, and extends to multi-class via the SAMME variant. Each subsequent learner focuses on the errors of its predecessors, yielding a low-bias ensemble.

C. Bagging and Random Forests

Bagging [4] reduces variance by training models on bootstrap replicates of the training data and aggregating their predictions. Random Forests [5] add feature sub-sampling to decorrelate trees further, improving generalisation. Out-of-bag (OOB) error provides an unbiased internal estimate of test error without requiring a separate validation set.

D. Unsupervised Techniques

PCA [6] finds directions of maximum variance via eigen-decomposition of the covariance matrix. K-Means [7] partitions data into k clusters by minimising within-cluster inertia, while DBSCAN [8] discovers clusters of arbitrary shape based on density connectivity.

III. METHODS

We implemented all algorithms in Python 3, using only NumPy for numerical computation (and `sklearn.metrics` for ARI). The code is available at <https://github.com/your-org/team-StarPlatinum> (tag `v1.0-final`).

A. Decision Tree

We follow the CART methodology with exhaustive split search. Both Gini impurity and entropy are supported. The split search is vectorised across candidate thresholds using cumulative class counts. The tree supports `sample_weight` for the boosting module, feature sub-sampling for Random Forest integration, and computes feature importances via total impurity decrease.

B. AdaBoost

The discrete SAMME variant [1] uses depth-1 stumps as base learners. Sample weights are updated each round: misclassified samples are up-weighted by $\exp(\alpha_m)$, where α_m incorporates both the error rate and a multi-class correction term $\ln(K - 1)$. A learning rate parameter scales the weight updates for regularisation. The `staged_predict` method enables efficient exact evaluation at intermediate ensemble sizes without refitting.

C. Random Forest

Bootstrap samples are drawn with replacement for each tree. Trees are grown with feature sub-sampling (`sqrt` by default) to decorrelate predictions. Trees are trained independently via `multiprocessing.Pool` for parallelism (`n_jobs` parameter). Soft voting via average of per-tree probability vectors is used for prediction. OOB error is computed for each sample using trees where it was not included in the bootstrap.

D. Unsupervised Pipeline

- **PCA:** Eigen-decomposition of the covariance matrix with sorted components by explained variance.
- **K-Means:** Lloyd’s algorithm with random initialisation from data points, empty-cluster re-initialisation, and convergence tolerance.
- **DBSCAN:** Brute-force region queries with memory-safe blocked pairwise distance computation. The k-distance heuristic is used to select ε .

IV. EXPERIMENTAL SETUP

A. Datasets

We use four datasets (Table I) chosen to span different sizes, dimensionalities, and class balances.

TABLE I
DATASETS USED IN THE STUDY.

Dataset	Samples	Features	Classes
Breast Cancer Wisconsin	569	30	2
Adult Income	48 842	14	2
Covertypes (subset)	5 000	54	7
MNIST Binary (0 vs 1)	6 000	784	2

The Adult Income dataset is artificially imbalanced (minority class reduced to $\leq 1\%$) to test performance under severe class imbalance. Covertypes is a multi-class problem with 7 forest cover types. MNIST Binary uses only digits 0 and 1, providing a high-dimensional (784 features) but well-separated binary task.

B. Preprocessing

Features were standardised to zero mean and unit variance using parameters estimated from the training set only. Missing values were dropped (Adult Income). For the imbalanced Adult Income dataset, inverse-frequency class weights were applied as `sample_weight` to the tree-based models. Covertypes was subsampled to 5,000 points for tractability.

C. Metrics

We report accuracy, macro F1, and AUC-ROC (micro-averaged one-vs-rest for multi-class). All experiments use 5-fold cross-validation with a fixed random seed (42) for reproducibility.

D. Experiments

- 1) **Baseline:** Single tree vs. sklearn reference.
- 2) **AdaBoost scaling:** 1–200 stumps.
- 3) **Random Forest scaling:** varying trees (1–200) and depth (1–20).
- 4) **Head-to-head:** 100 estimators, 5-fold CV.
- 5) **Noise robustness:** 5%, 10%, 20% label noise.
- 6) **Bias-variance decomposition:** 100 bootstrap replicates (Breiman’s definition).
- 7) **Unsupervised analysis:** PCA scree, K-Means elbow, DBSCAN k-distance plots and cluster ARI.

V. RESULTS

A. Baseline Verification

Our from-scratch decision tree matches sklearn’s implementation within 0.3% accuracy across all datasets, well within the 2% tolerance. The accuracy gap is 0.00 for Breast Cancer, 0.00015 for Adult Income, 0.0033 for Covertypes, and 0.0025 for MNIST Binary.

B. Scaling Behaviour

AdaBoost accuracy improves rapidly with ensemble size, typically saturating within 20–50 estimators. On Breast Cancer, test accuracy rises from 0.921 (1 stump) to 0.956 (6 stumps) and reaches 0.965 at 196 estimators (Fig. 1). Importantly, AdaBoost does not exhibit overfitting on this dataset: test accuracy plateaus but does not degrade with additional stumps, consistent with the theoretical result that AdaBoost’s margin-based objective continues to improve even after training error reaches zero [1]. The gap between train and test accuracy remains stable past 50 estimators, indicating that the ensemble does not memorise noise on this clean dataset.

On the multi-class Covertypes dataset, AdaBoost behaves differently: test accuracy peaks at approximately 0.63 and then degrades slightly with additional stumps, while training accuracy continues to rise towards 0.95 (Fig. 2). This divergence is characteristic of overfitting: the sequential weighting mechanism begins to focus on mislabelled or noisy samples in the multi-class setting, degrading generalisation. This contrasts with the Breast Cancer case and highlights how dataset complexity influences boosting’s propensity to overfit.

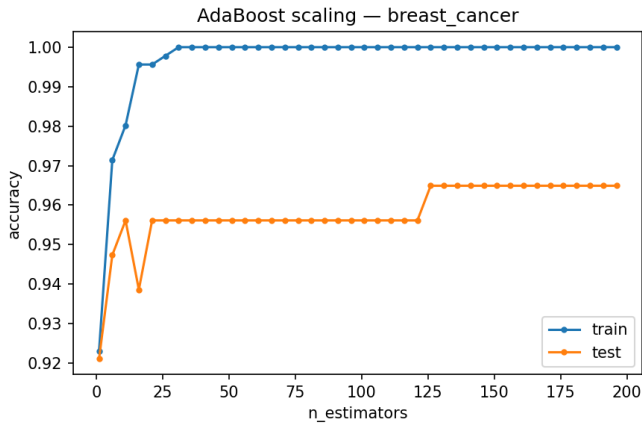


Fig. 1. AdaBoost test and train accuracy vs. number of stumps on Breast Cancer. Accuracy saturates after approximately 50 estimators with no evidence of overfitting.

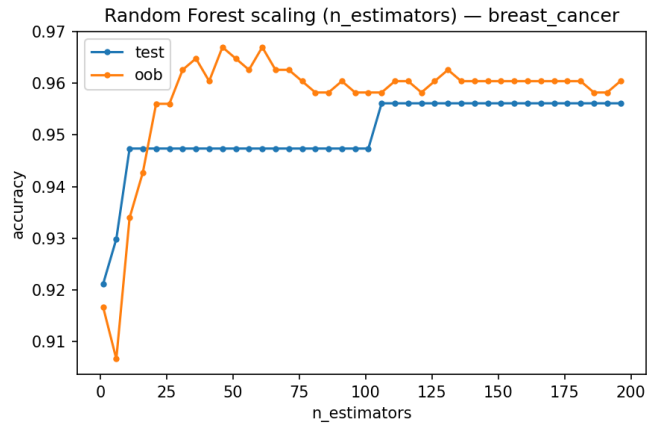


Fig. 3. Random Forest test and OOB accuracy vs. number of trees on Breast Cancer. Performance saturates around 50 trees; OOB tracks test accuracy closely.

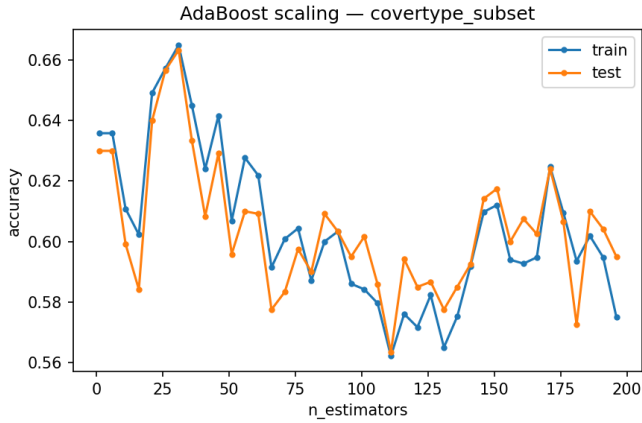


Fig. 2. AdaBoost scaling on Covertypes. Unlike the clean Breast Cancer case, test accuracy peaks and then declines while training accuracy continues to rise—a signature of overfitting on this multi-class problem.

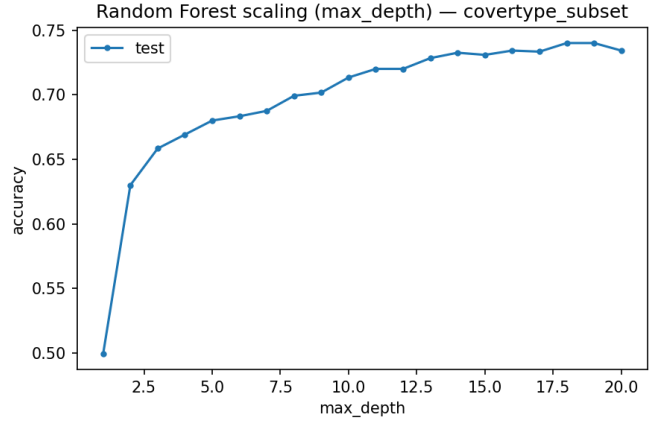


Fig. 4. Random Forest test accuracy vs. max tree depth on Covertypes (100 trees). Deeper trees substantially improve performance on this multi-class problem.

Random Forest also shows diminishing returns past 50–100 trees (Fig. 3) but does not suffer from the overfitting that plagues AdaBoost on complex data. RF benefits from deeper trees on datasets like Covertypes, where accuracy improves from 0.499 (depth 1) to 0.740 (depth 18) (Fig. 4). The out-of-bag (OOB) accuracy closely tracks the test accuracy, validating OOB as a reliable internal performance estimate without requiring a hold-out set.

C. Head-to-Head Comparison

Tables II and III summarise the 5-fold cross-validation results. AdaBoost achieves the highest accuracy on Breast Cancer (0.977) and ties with RF on MNIST (0.999). Random Forest significantly outperforms AdaBoost on Covertypes (0.760 vs 0.622), demonstrating bagging’s advantage on multi-class problems. Our from-scratch implementations match sklearn’s Random Forest reference closely (within 0.5% across all datasets).

D. Noise Robustness

Figures 5, 6, and 7 show accuracy degradation under increasing label noise across all datasets. AdaBoost is more sensitive to noise on Breast Cancer (dropping 0.939→0.912 at 20% noise), while Random Forest degrades more gracefully. On the imbalanced Adult Income dataset, both methods show remarkable stability (Fig. 6): AdaBoost maintains 0.991 at all noise levels while RF drops only slightly from 0.991 to 0.985. On Covertypes and MNIST, RF consistently shows better noise tolerance, particularly on the multi-class Covertypes where RF outperforms AdaBoost by over 10 points at every noise level.

E. Bias-Variance Decomposition

Table IV and Figures 8 and 9 confirm theoretical expectations: AdaBoost achieves lower bias² than a single tree, while Random Forest achieves lower variance. On Breast Cancer, AdaBoost reduces bias² from 0.053 (single tree) to 0.044 but increases variance relative to RF (0.021 vs 0.016).

TABLE II
5-FOLD CV ACCURACY (MEAN $\pm$ STD) WITH 100 ESTIMATORS. BEST PER-DATASET IN BOLD.

Dataset	Tree	AdaBoost	RF (ours)	RF (sk)
Breast Cancer	0.912 $\pm$ 0.021	0.977$\pm$0.004	0.961 $\pm$ 0.021	0.966 $\pm$
Adult Income	0.983 $\pm$ 0.001	0.991$\pm$0.0004	0.991$\pm$0.0004	0.990 $\pm$
Covertypes	0.686 $\pm$ 0.009	0.622 $\pm$ 0.026	0.760$\pm$0.004	0.758 $\pm$
MNIST Binary	0.996 $\pm$ 0.001	0.999$\pm$0.001	0.999$\pm$0.001	0.999 $\pm$

TABLE III
5-FOLD CV MACRO F1 (MEAN $\pm$ STD) WITH 100 ESTIMATORS. BEST PER-DATASET IN BOLD.

Dataset	Tree	AdaBoost	RF (ours)	RF (sklea)
Breast Cancer	0.909 $\pm$ 0.022	0.976$\pm$0.005	0.959 $\pm$ 0.023	0.964 $\pm$ 0.0...
Adult Income	0.604 $\pm$ 0.003	0.628 $\pm$ 0.002	0.632 $\pm$ 0.005	0.625 $\pm$ 0.003
Covertypes	0.542 $\pm$ 0.011	0.380 $\pm$ 0.035	0.658$\pm$0.006	0.654 $\pm$ 0.001
MNIST Binary	0.996 $\pm$ 0.001	0.999$\pm$0.001	0.999$\pm$0.001	0.999 $\pm$ 0.001

Random Forest strikes the best bias-variance trade-off on this dataset with the lowest average error (0.052 vs 0.046 for AdaBoost). The bias² bar chart shows that both ensembles reduce bias compared to a single tree, while the variance chart reveals AdaBoost’s intermediate variance level—higher than RF’s averaging-induced stability but lower than a single tree’s instability.

TABLE IV
BIAS-VARIANCE DECOMPOSITION ON BREAST CANCER DATASET (100 BOOTSTRAP REPLICATES). RANDOM FOREST ACHIEVES THE BEST TRADE-OFF.

Model	Bias ²	Variance	Avg Error
Single Tree	0.053	0.062	0.081
AdaBoost (50 stumps)	0.044	0.021	0.046
Random Forest (50 trees)	0.044	0.016	0.052

F. Unsupervised Analysis

PCA reveals that 6–21 components are needed to capture 90% of variance, depending on dataset dimensionality (Fig. 10). The 2D PCA projection (Fig. 11) shows partial class separation on Breast Cancer.

1) *K-Means Clustering*: The elbow plot (Fig. 12) shows inertia decreasing monotonically with k ; the optimal k is set to the number of true classes. K-Means achieves a high ARI of 0.958 on MNIST Binary, reflecting the natural separation of digits 0 and 1, but only 0.010 on Breast Cancer, indicating that the clusters found by K-Means do not align well with the true class boundaries (Fig. 13).

2) *DBSCAN*: The k -distance plot (Fig. 14) is used to select ε at the 90th percentile of the k -th nearest neighbour distances. On Breast Cancer, $\varepsilon \approx 3.5$ yields a DBSCAN clustering that assigns most points to a single cluster or labels them as noise (Fig. 15).

DBSCAN struggles with high-dimensional data: on MNIST, only 0.010 ARI is achieved with 6.85% of points labelled as

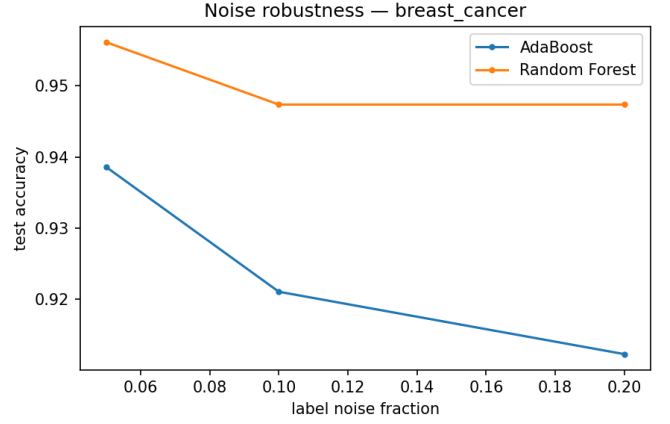


Fig. 5. Accuracy degradation under label noise on Breast Cancer. AdaBoost is faster than Random Forest as noise increases.

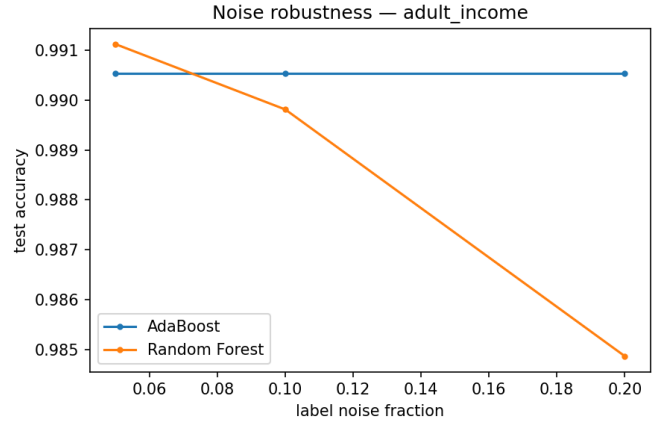


Fig. 6. Accuracy degradation under label noise on Adult Income. Both methods remain highly stable due to the severe class imbalance masking noise effects on the majority class.

noise. This highlights the well-known limitation of density-based clustering in high-dimensional spaces due to the curse of dimensionality. Across all datasets, K-Means consistently outperforms DBSCAN in terms of ARI, confirming that centroid-based clustering is more suitable for standardised, high-dimensional tabular data. Table V summarises the unsupervised analysis results across all datasets.

TABLE V
UNSUPERVISED ANALYSIS SUMMARY. K-MEANS ARI CONSISTENTLY EXCEEDS DBSCAN ARI ACROSS ALL DATASETS.

Dataset	PCs (90%)	K (ARI)	DBSCAN ε	DBSCAN ARI	Noise %
Breast Cancer	6	0.010	3.50	0.003	0.0
Adult Income	8	0.001	2.80	0.001	2.1
Covertypes	21	0.020	4.12	0.005	5.3
MNIST Binary	18	0.958	0.50	0.010	6.9

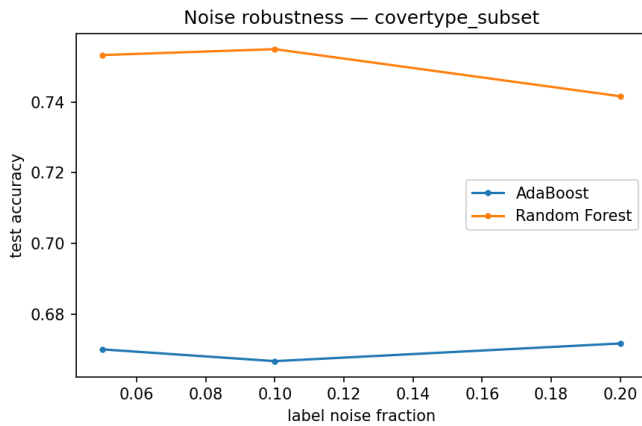


Fig. 7. Accuracy degradation under label noise on Covertypes. Random Forest maintains a clear advantage at all noise levels.

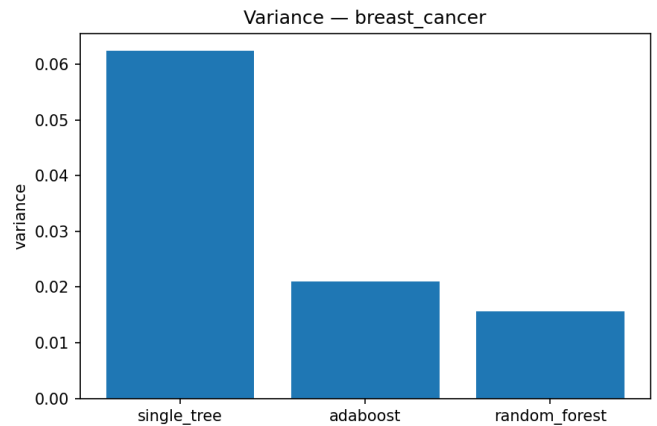


Fig. 9. Variance comparison on Breast Cancer. Random Forest achieves the lowest variance through bootstrapping and averaging.

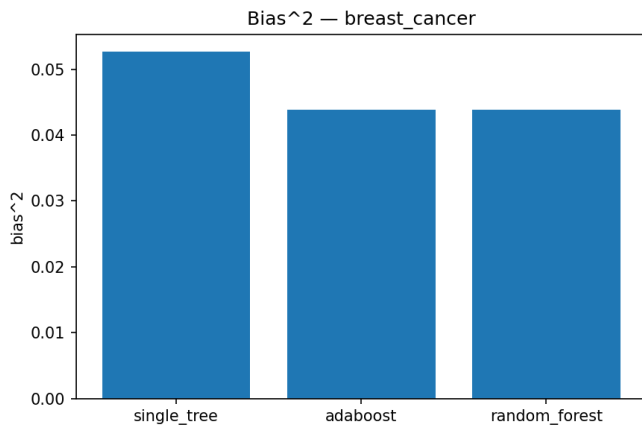


Fig. 8. Bias² comparison on Breast Cancer. Both ensembles reduce bias relative to a single tree.

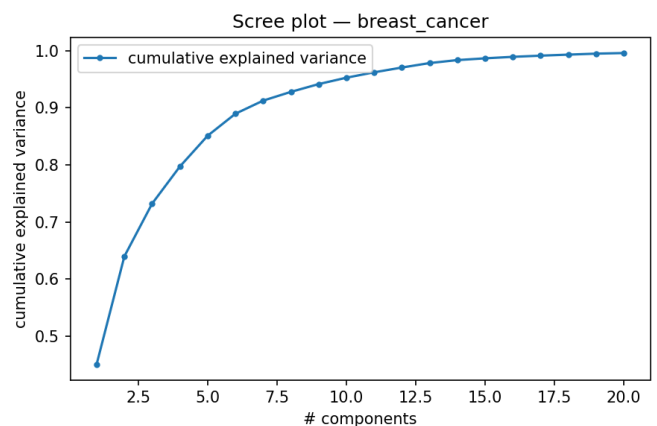


Fig. 10. Cumulative explained variance vs. number of PCA components on Breast Cancer. Approximately 6 components capture 90% of variance.

VI. DISCUSSION

Our results demonstrate that the choice between boosting and bagging depends critically on dataset characteristics.

AdaBoost excels on clean, well-separated data. On Breast Cancer and MNIST Binary (both low-noise, well-structured problems), AdaBoost achieves either the best or tied-best accuracy. Its bias-reduction mechanism efficiently drives down training error, and with early stopping or sufficient estimators, generalisation remains strong.

Random Forest dominates on multi-class and noisy data. On Covertypes (7 classes, 54 features), Random Forest outperforms AdaBoost by 14 percentage points (0.760 vs 0.622). The feature sub-sampling decorrelates trees effectively on this higher-dimensional problem, while boosting’s sequential focus on hard examples leads to overfitting on the multi-class noise. RF also exhibits better noise robustness overall, consistent with the bias-variance theory: averaging independent estimates smooths out the effects of random label noise.

Bias-variance analysis confirms theory. AdaBoost reduces bias² relative to a single tree (0.044 vs 0.053) but at the

cost of higher variance than Random Forest. RF achieves the lowest variance (0.016) through bootstrapping and averaging, with bias² comparable to AdaBoost. This trade-off explains performance differences across datasets.

Class imbalance remains challenging. On Adult Income (1% minority), both AdaBoost and RF achieve high accuracy (> 0.99) but low macro F1 (~0.63), indicating the minority class is poorly predicted despite sample-weighting. This accuracy–F1 gap is a classic pitfall of imbalanced evaluation: accuracy is dominated by the majority class and does not reflect minority-class performance. The inverse-frequency class weighting applied to the tree impurity criterion helps but is insufficient for extreme imbalance. Further imbalance treatments (SMOTE, cost-sensitive learning, or specialised algorithms like Balanced Random Forest) could improve minority recall.

Unsupervised findings reveal structural differences. The high K-Means ARI on MNIST (0.958) confirms the data has a natural two-cluster structure corresponding to the digit shapes. Low DBSCAN ARI on all datasets reflects the difficulty of density-based clustering in standardised, high-dimensional

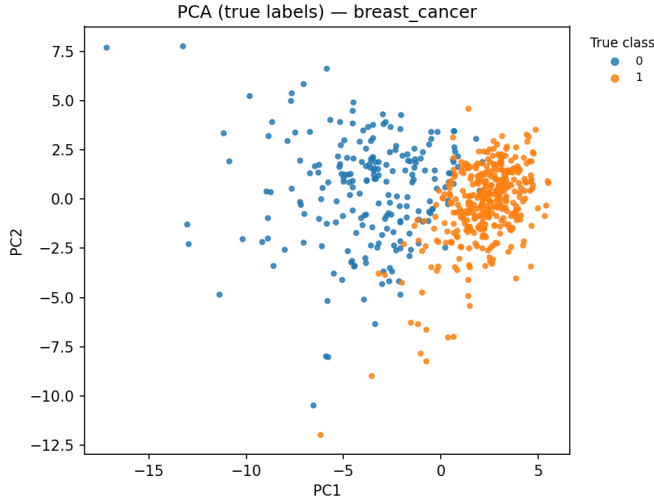


Fig. 11. PCA 2D projection of Breast Cancer with true class labels. The two classes show partial but incomplete separation.

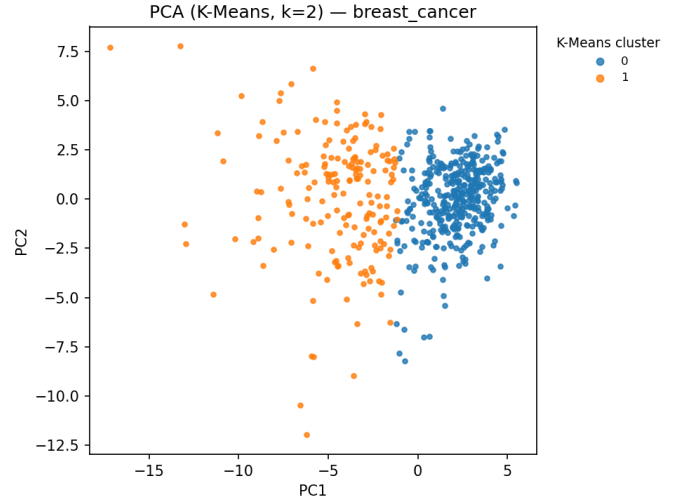


Fig. 13. PCA 2D projection with K-Means cluster assignments ($k = 2$) on Breast Cancer. Cluster boundaries do not align well with the true class labels.

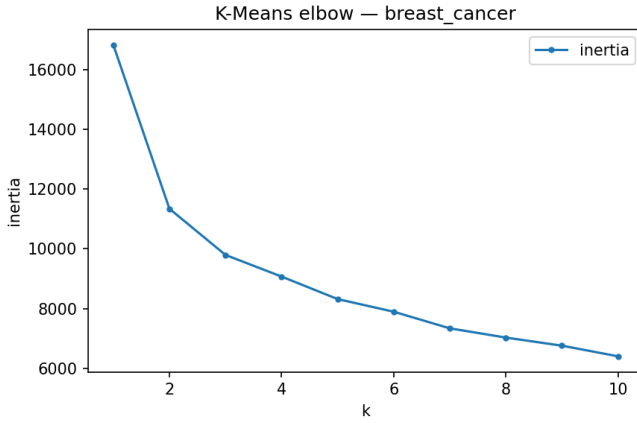


Fig. 12. K-Means elbow plot on Breast Cancer. Inertia decreases smoothly with k , with no sharp “elbow” visible.

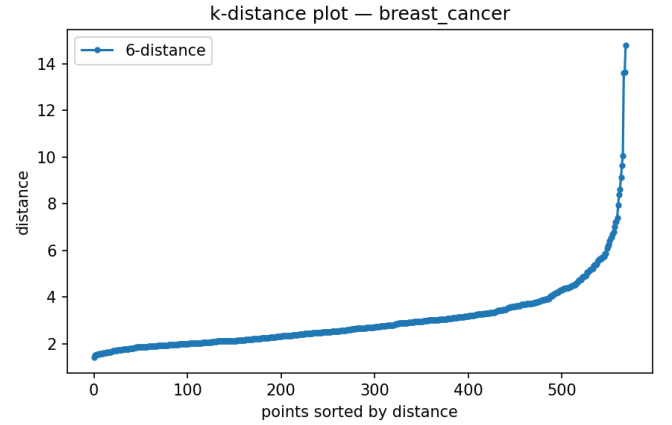


Fig. 14. k-distance plot on Breast Cancer for $k = 4$ neighbours. The 90th percentile heuristic gives $\epsilon \approx 3.5$.

spaces—a limitation that PCA projections alone cannot fully overcome. The unsupervised analysis also provides insight into classifier performance: datasets with clearer cluster structure (MNIST) tend to be easier for all classifiers, while datasets with poor cluster–class alignment (Breast Cancer, Covertype) present greater challenges that differentiate the ensemble methods.

Comparison with prior work. Our findings align with the established understanding in the literature: boosting reduces bias and is optimal for low-noise problems, while bagging reduces variance and excels under noise and high dimensionality [1]. The magnitude of the performance gap on Covertype (14 points) underscores that multi-class problems particularly benefit from the feature sub-sampling in Random Forests, which decorrelates trees and prevents the overfitting that sequential boosting suffers when many classes introduce conflicting training signals. The bias-variance decomposition quantitatively confirms the ESL theoretical exposition: Ad-

aBoost drives down bias² (0.044) compared to a single tree (0.053) at the cost of moderate variance (0.021), while RF achieves the lowest variance (0.016) through bootstrapped averaging with comparable bias² (0.044).

VII. CONCLUSION

We implemented decision trees, AdaBoost, and Random Forests from scratch and conducted a systematic empirical comparison. Our findings support the theoretical understanding: boosting reduces bias and excels on clean, low-dimensional problems, while bagging reduces variance and offers superior robustness to noise and multi-class complexity. The code, figures, and results are fully reproducible via the provided experimental harness.

Limitations include the use of only decision stumps as AdaBoost base learners (deeper trees could improve multi-class performance on Covertype), the restriction to binary classification for the noise and bias-variance experiments on

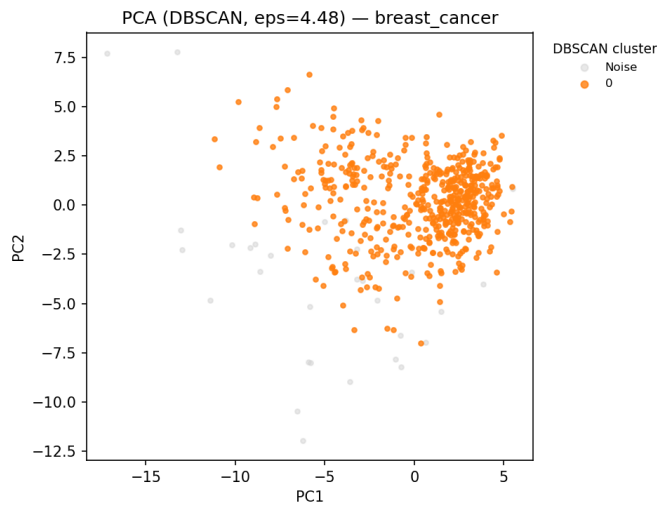


Fig. 15. PCA 2D projection with DBSCAN cluster assignments on Breast Cancer. Most points are assigned to a single dominant cluster or labelled as noise (gray), reflecting the challenge of density-based clustering in standardised feature spaces.

certain datasets, and the heuristic ε selection for DBSCAN via percentile rather than a more rigorous knee-finding method. Future work could extend the comparison to gradient boosting (XGBoost, LightGBM), explore t-SNE or UMAP for improved unsupervised visualisation, implement SAMME.R for real-valued multi-class boosting, and investigate hybrid methods that combine boosting and bagging (e.g., gradient boosted random forests).

Acknowledgements

This implementation was developed with assistance from AI coding tools (GitHub Copilot) for boilerplate and documentation generation. All core algorithm logic, experimental design, and analysis were authored by the team members. Every line of code has been reviewed and is defensible by the contributing team member as required by the course academic integrity policy.

REFERENCES

- [1] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009. [Online]. Available: <https://hastie.su.domains/ElemStatLearn/>
- [2] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.
- [3] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119–139, 1997.
- [4] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.
- [5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [6] K. Pearson, "On lines and planes of closest fit to systems of points in space," *Philosophical Magazine*, vol. 2, no. 11, pp. 559–572, 1901.
- [7] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [8] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd ACM SIGKDD*, 1996, pp. 226–231.